CrisprClamp User Manual

Yoon Sing Yap

2026-10-20

1. What CrisprClamp Does

CrisprClamp (CRISPR Combinatorial Library Admission and Memory-bounded Pair counter) counts pairs of guide RNAs from paired-end sequencing reads. It is designed for combinatorial dual-guide CRISPR screens. It reads both gzip-compressed (.fastq.gz) and uncompressed (.fastq) files directly.

CrisprClamp uses exact sequence matching. Every base in the extracted read window must match the reference guide sequence. Guides must appear at fixed coordinate positions within each sequencing read.

CrisprClamp does not calculate statistical scores, p-values, or gene-level enrichment. Use downstream software such as MAGeCK for statistical testing. CrisprClamp does not allow nucleotide mismatches during counting. It does not align reads across variable offsets or trim adapters.

---

2. What You Need

To run CrisprClamp, you need: - An Apple Silicon Mac. - Rust. Section 3 shows how to install it. - Git. It is used to get the source code. - Python 3. It is used for the table export. - Your FASTQ files and your guide tables.

---

3. Install

Build CrisprClamp from source:

1. **Install Rust**: Visit the official Rust installation page at rust-lang.org and follow the instructions to install the Rust toolchain.
2. **Get the source**:

```
git clone https://github.com/Yoonecorn/crisprclamp.git
cd crisprclamp
```

3. **Build the release executable**:

```
cargo build --release --locked
```

4. **Test that the program runs**:

```
./target/release/crisprclamp --help
```

CrisprClamp was tested on Apple Silicon macOS with Rust 1.98.0.

- **Repository**: https://github.com/Yoonecorn/crisprclamp
- **License**: MIT OR Apache-2.0. You may use either license.

---

4. Quick Start in Four Steps

Follow these four steps to run CrisprClamp on the included tiny example dataset from within the cloned repository folder.

Step 1: Prepare Three TSV Files

The three tab-separated text files are already provided in examples/tiny_dataset. For your own screens, create three tab-separated text files without header lines.

guides_a.tsv (Guide A names and sequences):

```
G01 ACGTACGTACGTACGTACGT
G02 CGTACGTACGTACGTACGTA
G03 GTACGTACGTACGTACGTAC
G04 TACGTACGTACGTACGTACG
G05 TTTTCCCCAAAAGGGGTTTT
```

guides_b.tsv (Guide B names and sequences):

```
H01 AAAACCCCGGGGTTTTAAAA
H02 CCCCGGGGTTTTAAAACCCC
H03 GGGGTTTTAAAACCCCGGGG
H04 TTTTAAAACCCCGGGGTTTT
H05 ACACACACACACACACACAC
```

constructs.tsv (Expected guide combinations):

```
C01 G01 H01
C02 G02 H02
C03 G03 H03
C04 G04 H04
C05 G05 H05
```

All sequences in a guide file must have the same length (for example, 20 bases).

Step 2: Run the Counting Command

Run this command in your terminal:

```
./target/release/crisprclamp \
  --mode dual \
  --sample sample_tiny \
  --fastq examples/tiny_dataset/R1.fastq.gz \
  --fastq-r2 examples/tiny_dataset/R2.fastq.gz \
```

```
  --guides-a examples/tiny_dataset/guides_a.tsv \
  --guides-b examples/tiny_dataset/guides_b.tsv \
  --constructs examples/tiny_dataset/constructs.tsv \
  --a-start 0 \
  --b-start 0 \
  --b-source R2 \
  --memory-budget-bytes 2 147 483 648 \
  --output ./tiny_output
```

The --memory-budget-bytes 2 147 483 648 option sets a 2-gigabyte memory planning limit to verify that the system has enough memory before reading sequencing files. The planning estimate depends on the budget setting, and a value close to the budget is not a problem on its own. For large FASTQ files, you can add --workers 4 to use four workers. On the tiny example, the counts were the same.

The program displays the memory plan on your screen:

```
resource.bounded_dual_plan={"common":{"bloom":256,...},"estimate_bytes":2 135 743 939,...}
```

The program also prints timing lines.

Step 3: Open the Results

CrisprClamp writes result files into the ./tiny_output directory. The primary counts are written to raw-counts.jsonl. Each line records one construct ID and its observed read count:

```
{"counts":[2],"row_id":"C01",...}
{"counts":[1],"row_id":"C02",...}
{"counts":[0],"row_id":"C03",...}
{"counts":[0],"row_id":"C04",...}
{"counts":[0],"row_id":"C05",...}
```

Constructs with zero matching reads remain in the file with a count of 0.

Step 4: Export a Count Table

Convert raw-counts.jsonl into a standard tab-separated table using Python 3 and the provided script:

```
python3 examples/tiny_dataset/export_table.py \
  --constructs examples/tiny_dataset/constructs.tsv \
  --jsonl ./tiny_output/raw-counts.jsonl \
  --output counts.tsv
```

Output:

```
Exported 5 rows to counts.tsv
```

View the exported counts.tsv file:

```
construct_id    guide_a guide_b count
C01 G01 H01 2
```

```
C02 G02 H02 1
C03 G03 H03 0
C04 G04 H04 0
C05 G05 H05 0
```

---

5. How to Find Your Own Settings

To run CrisprClamp on your own sequencing library, determine the starting position of each guide:

- **--a-start**: The 0-based position where Guide A begins in Read 1. If Guide A begins at the very first base of Read 1, set `--a-start 0`. If Read 1 has a 10-base prefix before Guide A, set `--a-start 10`.
- **--b-start**: The 0-based position where Guide B begins in its read. If Guide B begins at the first base of Read 2, set `--b-start 0`.
- **--b-source R2**: Tells CrisprClamp to read Guide B from Read 2. If this option is omitted, the tool reads both guides from Read 1.

All guides in your library table must have the exact same nucleotide length.

---

6. Read Your Results

CrisprClamp creates an output directory with summary and count files.

Output Files

Filename	Description
raw-counts.jsonl	Read counts for every construct in the reference library.
sample-qc.jsonl	Quality control summary and total counts for each read class.
terminal-totals.jsonl	Detailed read counts across all twelve internal categories.
discovery.jsonl	Observed sequences from unmapped read pairs.
run-metadata.jsonl	Execution times, hardware information, and run parameters.
completion.jsonl	Receipt confirming that the run finished successfully.

The Five Read Classes

Every read pair is placed into exactly one of five mutually exclusive classes:

Read Class	Meaning
Mapped	Both extracted guides match an expected construct in your reference list.
Recombinant	Both guides match the library, but the pair is not in your construct list.
Ambiguous	A sequence matches multiple guides or multiple extraction positions.
Unmapped	One or both extracted sequences do not match any guide in the library.
Invalid	A read contains an 'N' base, non-ACGT characters, or is shorter than the guide.

Every read pair goes into exactly one class. The sum of all five classes equals the total number of read pairs in your FASTQ files.

---

7. Common Problems and Fixes

If CrisprClamp stops with an error code, use this table to resolve the problem:

Error Code	Reason	How to Fix It
preflight.output_exists	The output directory already exists.	Choose a new output name (for example, --output ./tiny_output_run2). Never delete or overwrite previous result folders.
preflight.output_parent	The output path does not specify a parent directory (such as ./).	Add ./ before the folder name (for example, --output ./tiny_output).
resource.memory_budget	The estimated memory exceeds --memory-budget-bytes.	Increase --memory-budget-bytes (for example, set --memory-budget-bytes 4 294 967 296 for 4 GiB).
preflight.unknown_a_guide	A construct references a Guide A name not in guides_a.tsv.	Check the spelling of guide names in constructs.tsv and guides_a.tsv.
preflight.unknown_b_guide	A construct references a Guide B name not in guides_b.tsv.	Check the spelling of guide names in constructs.tsv and guides_b.tsv.

Error Code	Reason	How to Fix It
preflight.duplicate_construct_id	A construct ID is listed more than once.	Remove duplicate construct lines from constructs.tsv.
preflight.duplicate_pair	The same guide pair is assigned to more than one construct.	Remove the duplicate guide combination from constructs.tsv.
preflight.guide_alphabet	A guide sequence contains letters other than A, C, G, or T.	Remove invalid characters or spaces from the guide sequence.
preflight.guide_length	Guide sequences in a table do not all have the same length.	Ensure all guide sequences in the file have the exact same number of bases.

8. Large Libraries

CrisprClamp handles up to 50 million constructs in one run. Larger libraries need extra setup. Open an issue on GitHub for help.

---

9. Glossary

- **Construct**: An expected combination of Guide A and Guide B.
- **FASTQ**: A standard text file format for sequencing reads and quality scores.
- **Guide RNA**: A short nucleotide sequence (typically 20 bases) targeting DNA.
- **Invalid**: A read pair rejected due to 'N' bases, bad characters, or short length.
- **Mapped**: A read pair where both guides match a target construct.
- **Read Pair**: Two matched reads (Read 1 and Read 2) from paired-end sequencing.
- **Recombinant**: A read pair with two known guides that do not form a listed construct.
- **TSV**: A tab-separated text file format.
- **Unmapped**: A read pair where one or both guides do not match the reference library.

---

10. Where to Learn More

For technical details and validation tests, read the CrisprClamp manuscript. It is the file docs/CrisprClamp_manuscript.pdf in the repository.

- **Repository**: https://github.com/Yoonecorn/crisprclamp
- **Contact**: Open an issue at https://github.com/Yoonecorn/crisprclamp/issues